# Laboratorium 1 – parser strony WWW (poziom 5.0)

Rola PARSER: scraper serwisu z infinite scrollem i podstronami szczegółów (Selenium + BeautifulSoup). Dane: 400 rekordów, 33 atrybuty. Analiza i wnioski – poniżej.

## 1. Użyta technologia i uzasadnienie wyboru

**Python 3.11** – zalecany w instrukcji (requests + BeautifulSoup, pandas + matplotlib, Selenium na poziomie 5.0). **Selenium 4 + Chrome** – serwis wstrzykuje rekordy przez *fetch* po przewinięciu strony (IntersectionObserver), a dane dodatkowe podstrony pojawiają się dopiero po scrollu i po kliknięciu „Read more”; zwykłe *requests* widzi tylko pierwszych 10 kart (sekcja 6). **BeautifulSoup + lxml** – ekstrakcja z gotowego drzewa DOM jest dużo szybsza niż setki wywołań *find_element* przez protokół WebDriver. **WebDriverWait** z warunkiem zamiast *sleep* – czekamy na konkretny stan strony (przybyło kart, pojawił się wiersz tabeli), a nie na zgadywany czas; serwis losuje opóźnienie 0–3 s. **pandas** – czyszczenie, deduplikacja i eksport (CSV, XLSX, JSON); **matplotlib** – wykresy; **psutil** – pomiar pamięci; **pytest** – testy jednostkowe.

## 2. Opis serwisu (rola AUTOR – serwis partnera)

Opis sporządzony na podstawie kodu serwisu partnera (katalog `PJN-main`). *Sposób generowania danych i uzasadnienie wyboru utrudnień uzupełnia autor serwisu.*

| Element | Opis |
|---|---|
| Stos | Node.js + Express 5, szablony EJS, dane w `database/data.json` (400 rekordów); uruchamiany w Dockerze lub `node app.js`. |
| Trasy | `GET /feed` (lista), `GET /posts/:id` (szczegóły), API: `/api/v1/feed?limit&offset`, `/api/v1/posts/:id/additional-info`, `/api/v1/posts/:id/read-more`. |
| Lista | 10 kart renderowanych serwerowo (klasy `post_title`...), kolejne porcje po 10 doładowywane przez JS przy scrollu (klasy `post-title`...) – **dwie różne struktury tych samych rekordów**. |
| Podstrona | Podstawowe pola w HTML; „Specifications” (tabela, kolejność wierszy losowa) i „Product details” (lista) ładowane po przewinięciu do sentinela; sekcja „Read more” po kliknięciu przycisku, w jednej z 3 losowych kolejności elementów. |
| Utrudnienia | ceny w 6 walutach (PLN, EUR, USD, GBP, CZK, JPY) zapisywane symbolem lub kodem ISO, przed lub po liczbie, z separatorem tysięcy lub bez; daty m.in. ISO, dd.mm.yyyy, „20 August 2025”, „Aug 20, 2025”, dwuznaczne dd/mm/yyyy vs mm/dd/yyyy oraz względne („3 days ago”, „Today”); braki w 7 polach (9–26% rekordów); encje ( , ó, ') i znaczniki HTML w opisach; losowe opóźnienie 0–3 s każdego żądania; doładowywanie porcjami, atrybuty po interakcji (scroll, klik), losowa kolejność atrybutów. |

## 3. Opis parsera

**Przepływ.** (1) Faza listy: otwarcie `/feed`, pętla „przewiń do końca → *WebDriverWait* aż przybędzie kart albo pojawi się komunikat o końcu” aż do wyczerpania rekordów (40 porcji po 10 kart). (2) Faza podstron: dla każdego id otwarcie `/posts/:id`, przewinięcie do sentinela (czekanie na wiersze tabeli), kliknięcie „Read more” (czekanie na elementy z sekcji), pobranie `page_source` i ekstrakcja BeautifulSoup. (3) Łączenie danych listy i podstron po id, czyszczenie, deduplikacja, walidacja, eksport.

**Wymagania obowiązkowe 5.0:** Selenium; jawne oczekiwanie (WebDriverWait z warunkiem); obsługa doładowywania do wyczerpania; ponawianie z opóźnieniem wykładniczym 1, 2, 4, 8 s (limit 5 prób i 30 s – testowane jednostkowo); porównanie HTML vs DOM (sekcja 6). Opóźnienie między żądaniami ≥ 0,5 s zapewnia wspólny dla wszystkich przeglądarek limiter.

**Wybrane rozszerzenia** (wymagane co najmniej 2; zrealizowano 5 z listy):

| Rozszerzenie | Realizacja |
|---|---|
| Zrzut ekranu i źródło przy błędzie | `_diagnose()` zapisuje PNG i HTML do `output/…/errors/`; pojedynczy błąd nie przerywa całości, a niezdrowa przeglądarka jest zastępowana nową. W pełnym przebiegu błędów było 0, więc katalog nie powstał. |

| Rozszerzenie | Realizacja |
|---|---|
| Tryb headless + pomiar czasu | Opcja `--headless`; pomiar w obu trybach w sekcji 7. |
| Wznawianie (checkpoint) | Lista w `checkpoint_feed.json`, każda podstrona dopisywana do `checkpoint_details.jsonl` po pobraniu; ponowne uruchomienie pomija gotowe id; `--fresh` zeruje. |
| Wykrycie i użycie źródła JSON | Strona woła `/api/v1/…` (wykryte z Resource Timing, `compare_raw_vs_dom.py`); `scrape_api.py` daje identyczny wynik w ułamku czasu (sekcja 7), z obsługą 429/Retry-After. |
| Czas i pamięć: bez przeglądarki vs Selenium | `benchmark.py` – ten sam podzbiór, wyniki w sekcji 7. |
| Dodatkowe (poziom 4.0) | równoległe przeglądarki z limitem żądań (`--workers`), CLI (adres, limit, plik wyjściowy), logowanie do pliku z podsumowaniem, eksport CSV+XLSX+JSON, 42 testów jednostkowych, walidacja typów z raportem odrzuceń. |

**Selektory** – opierają się na semantycznych klasach i id, nigdy na pozycji elementu (kolejność jest losowa):

| Dane | Selektor / metoda | Dlaczego |
|---|---|---|
| karta listy | `#feed article.post` | wspólna dla kart serwerowych i doładowanych |
| data, miejsce, kategoria | `span` z klasą regex `^post[_-](published_)?date$` itd. | SSR używa *post_**, JS – *post-**; regex łapie oba warianty |
| id rekordu | `.read-more a[href]` + regex `/posts/(\d+)` | karty nie mają *data-id*; URL jest kontraktem; awaryjnie pozycja w feedzie |
| cena, opis, polubienia | `div.price`, `div.description`, `.interaction .likes` | semantyczne klasy |
| tabela specyfikacji | `#specifications tr`: klucz z tekstu `th`, wartość z `td` | kolejność wierszy losowa → mapowanie po etykiecie |
| lista „Product details” | `#extra-list li`, podział na pierwszym „:” | URL zawiera dwukropek |
| sekcja Read more | `#read-more-content .summary / .description / .author / .post_published_at` | 3 losowe kolejności; zawężenie do kontenera, bo *.description* występuje też w karcie głównej |
| oczekiwania | `#specifications tr`, `#read-more-button` (clickable), `#feed-status` | warunek stanu strony zamiast czasu |

**Odporność selektorów na zmianę wyglądu serwisu.** Przetrwałyby zmiany czysto wizualne (CSS, kolory, układ, kolejność elementów), bo nie używamy pozycji, klas stylistycznych ani XPath-ów po indeksach: selektory po *id* (`#specifications`, `#read-more-content`), id z adresu URL oraz mapowanie po etykietach. Są najbardziej odporne: id z URL i pola wyciągane po kluczu. Nie przetrwałyby zmiany nazw klas lub znaczników: `article.post`, h2 dla tytułu, `div.price`, regexy `post[_-]…` (już raz zepsute przez autora między SSR a JS, wymagały obsługi dwóch wariantów); najkruchsze są: etykiety w `th` (np. „Brand” – zmiana języka UI rozbiłaby mapowanie), teksty komunikatów końca listy („reached the end”, „No more posts”) oraz prefiksy „Date:”, „Author:”. Ryzyko ogranicza fallback: jeśli komunikatu nie ma, pętla kończy się po przekroczeniu limitu oczekiwania na nową kartę (przy braku postępu), a brak pola daje pustą wartość, nie wyjątek.

# 4. Wyniki kontroli jakości danych

Pozyskano **400 rekordów** (400 na liście, 400 podstron; 0 błędów, 0 ponowień) w **864 s**, 1241 żądań, 40 porcji doładowania. Kolumn: 33.

**Braki wartości** (puste pole = brak danych w serwisie, nie błąd parsera; pozostałe 26 kolumn kompletne):

| Kolumna | Braki [szt.] | Braki [%] |
|---|---|---|
| brand | 95 | 23,8% |
| location | 51 | 12,8% |
| rating | 37 | 9,2% |
| availability | 70 | 17,5% |
| condition | 102 | 25,5% |
| seller | 87 | 21,8% |
| author | 76 | 19,0% |

**Duplikaty:** przed odrzuceniem 400 rekordów, duplikatów po id 0, po zawartości (bez id i sku) 0; po odrzuceniu 400. Serwis nie zawiera duplikatów (autor nie wybrał tego utrudnienia), mechanizm deduplikacji został jednak sprawdzony testem jednostkowym na danych syntetycznych.

**Typy kolumn:** kolumny liczbowe (price, price_pln, likes, comments, rating, tags_count) są liczbowe – wszystkie; liczba odrzuconych (nieparsowalnych) wartości: 0. Waluty: CZK 76, EUR 73, JPY 68, USD 65, GBP 60, PLN 58. Jakość dat (pole published): dokładne 346, dwuznaczne 52, względne 2.

**Wartości odstające** (reguła 1,5·IQR): price: 0; price_pln: 0; likes: 0; comments: 0; rating: 0. Test poprawności obsługi przecinka: cyfry surowej ceny porównane z parsowaną liczbą ×100 – 0 niezgodności; wartości powyżej 100 000 (typowy objaw „129900 zamiast 1299”): 0. Maksymalna cena nominalna to 2 499,48, więc nie ma ani anomalii w danych, ani błędu skali po stronie parsera. Duża rozpiętość cen w PLN (0,52–12 374,75) nie jest anomalią: wynika z mieszania walut (JPY i CZK mają niski kurs), a kwoty nominalne są niezależne od waluty.

**Spójność lista–podstrona:** 0 rozbieżności tytułu/kategorii/lokalizacji.

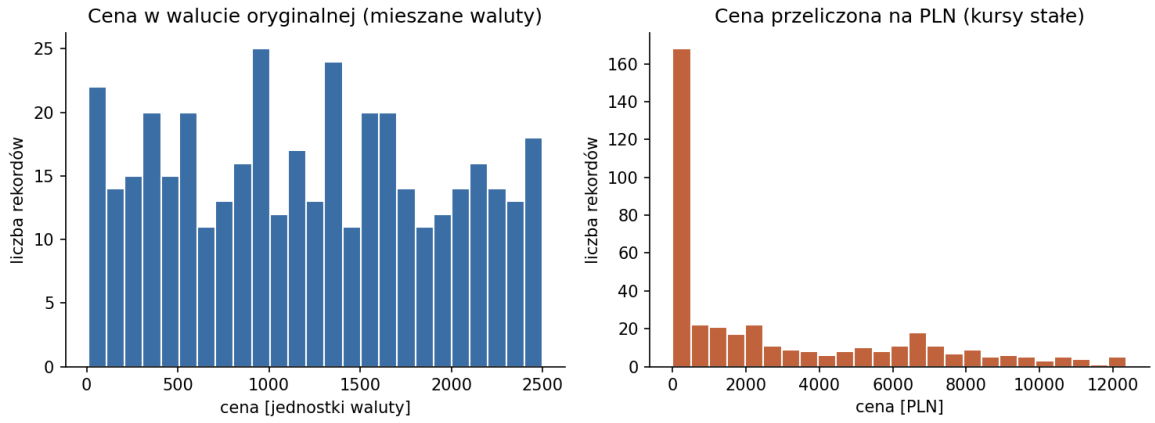
**Weryfikacja względem pliku źródłowego** (dostępnego tylko w naszym środowisku testowym; w realnym scrapingu go nie ma): porównano 400 rekordów × 17 pól – **0 niezgodności**. Dodatkowo zbiór z Selenium jest identyczny co do komórki ze zbiorem z endpointu JSON (400×33).

**Wniosek i poprawki.** Analiza potwierdza poprawność parsera: brak błędów skali cen, kompletne kolumny liczbowe, zgodność listy z podstroną i ze źródłem. Z kontroli i testu na 12 rekordach wynikały decyzje: (a) kolumna *url* z listy „Product details” okazała się w 100% pusta (serwis nie zwraca tego pola) – usunięto ją; (b) placeholdery serwisu („Not available”, „Unknown”, „Date unknown”) zamieniane są na puste wartości, żeby nie zafałszować statystyk – dotyczy to też marki „Unknown”, którą traktujemy jako brak; (c) pola tekstowe wstawiane przez *textContent* zawierają literalne encje ( , '), więc czyszczenie dekoduje encje i usuwa znaczniki (np. <*strong*> w opisach); (d) 52 dat w formacie z ukośnikiem jest realnie dwuznaczne (w danych występują i dd/mm, i mm/dd) – przyjęto dd/mm z flagą *date_quality* = *ambiguous* zamiast cichego zgadywania.

# 5. Statystyki opisowe i wnioski

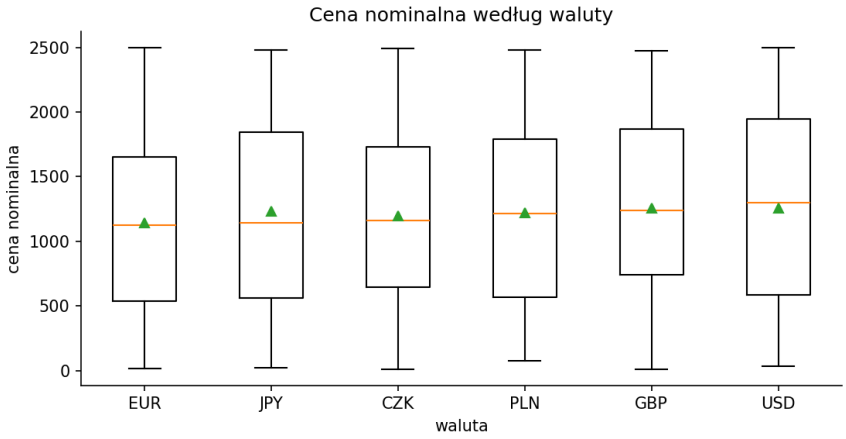
| Cecha | n | średnia | mediana | dominanta* | Q1 | Q3 | min | max | odch. std. |
|---|---|---|---|---|---|---|---|---|---|
| cena (nominalna) | 400 | 1 214,86 | 1 203,33 | 881–1 006 | 572,05 | 1 789,50 | 10,09 | 2 499,48 | 721,09 |
| cena [PLN] | 400 | 2 837,05 | 1 264,88 | 0–619 | 157,56 | 5 280,88 | 0,52 | 12 374,75 | 3 344,91 |
| polubienia | 400 | 24 546,80 | 24 023,00 | 45 003–47 491 | 12 053,75 | 38 288,50 | 219,00 | 49 979,00 | 14 959,24 |
| komentarze | 400 | 1 264,94 | 1 253,50 | 761–885 | 651,75 | 1 915,50 | 18,00 | 2 494,00 | 726,19 |
| ocena | 363 | 3,04 | 3,00 | 2,3 | 2,10 | 4,00 | 1,00 | 5,00 | 1,11 |
| liczba tagów | 400 | 2,43 | 2,00 | 1,0 | 1,00 | 3,00 | 1,00 | 4,00 | 1,12 |

** dla cech ciągłych wszystkie wartości są unikalne, więc dominanta liczona jest jako najliczniejszy przedział histogramu (20 przedziałów); dla oceny i liczby tagów – wartość najczęstsza.*



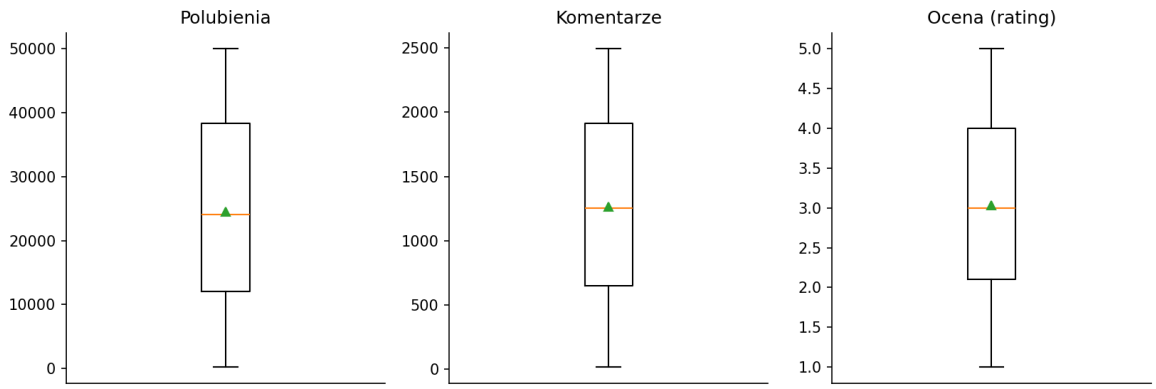

*Rys. 1. Histogram ceny: nominalnej (lewo) i w PLN po przeliczeniu stałymi kursami (prawo).*

Ceny nominalne mają rozkład zbliżony do jednostajnego (10–2 499), średnia 1 215 ≈ mediana 1 203. Po przeliczeniu na PLN rozkład jest silnie prawoskośny (średnia 2 837 ≫ mediana 1 265), z dużym skupiskiem przy niskich wartościach: 190 z 400 rekordów w pierwszym przedziale 0–1 031 zł (JPY i CZK nie przekraczają 500 zł, GBP/EUR/USD sięgają 10–12 tys. zł). To skutek mieszania walut i niezależnego losowania kwoty, a nie błąd parsera – wniosek: porównywanie cen bez przeliczenia jest bezzasadne, a wartości w PLN zależą od przyjętych (stałych, przybliżonych) kursów.



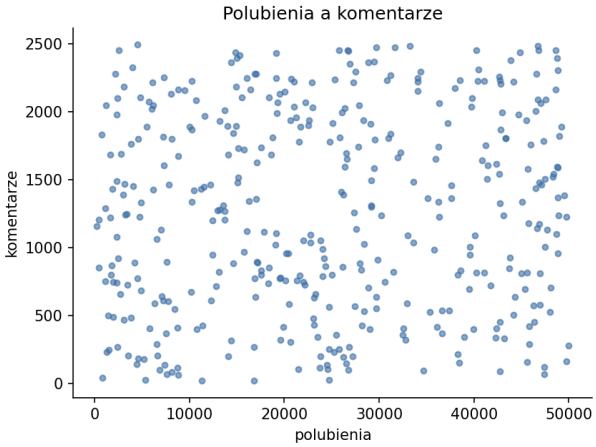

*Rys. 2. Wykres pudełkowy ceny nominalnej według waluty (◦ – średnia).*

Mediany cen nominalnych są podobne we wszystkich walutach (od 1 123 do 1 296) – dodatkowy dowód, że parser nie zniekształca skali dla żadnej waluty (np. JPY bez miejsc po przecinku czy zapis z separatorem tysięcy).



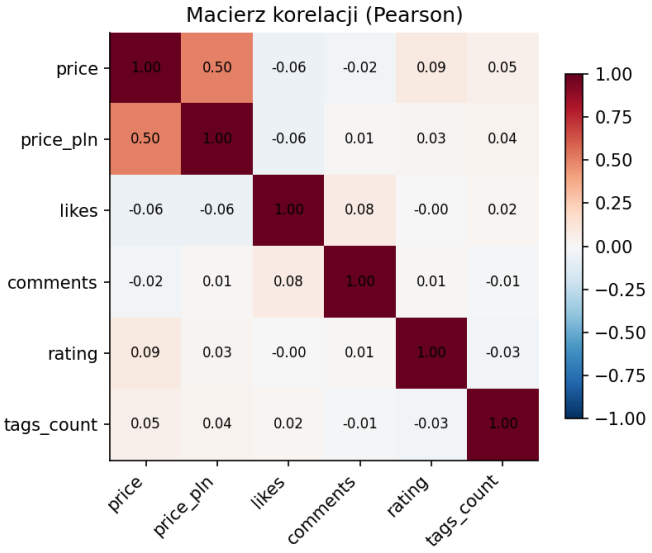

*Rys. 3. Wykresy pudełkowe: polubienia, komentarze, ocena (◦ – średnia).*

Polubienia (219–49 979), komentarze (18–2 494) i oceny (1,0–5,0) są równomiernie rozłożone, mediana ≈ średnia, brak wartości odstających. Ocena ma 37 braków (9,2%) – analizowane n = 363.




*Rys. 4. Zależność polubień i komentarzy.*

**Korelacja.** polubienia–komentarze: Pearson 0,080, Spearman 0,082 (n = 400); cena–ocena: 0,087; cena PLN–polubienia: -0,057; ocena–polubienia: -0,004. Wszystkie współczynniki są praktycznie zerowe. W prawdziwym serwisie oczekiwalibyśmy wyraźnie dodatniej zależności polubienia–komentarze (popularne posty zbierają jedno i drugie), a więc jej brak nie ma sensu merytorycznego – wskazuje, że dane zostały wygenerowane losowo i niezależnie. Wykluczyliśmy błąd parsera: wartości zgadzają się ze źródłem w 100%. Pozostałe pary (cena–ocena itd.) nie mają uzasadnienia merytorycznego dla zależności, więc wynik ≈ 0 jest tu zgodny z oczekiwaniem.




*Rys. 5. Macierz korelacji cech liczbowych.*

# 6. Surowy HTML a drzewo DOM po wykonaniu skryptów (wymóg 5.0)

Surowy HTML pobrany *requests*, bez przeglądarki, i DOM po wykonaniu JS w Selenium (podstrona nr 7):

| Co liczymy | Surowy HTML | DOM po JS |
|---|---|---|
| karty rekordów na `/feed` | 10 | 400 (po scrollu do końca; zaraz po załadowaniu: 10) |
| wiersze tabeli specyfikacji (podstrona) | 0 | 5 |
| pozycje listy „Product details” | 0 | 3 |
| elementy sekcji „Read more” | 0 | 4 |
| rozmiar HTML [B] lista / podstrona | 17654 / 3897 | — |

**Wniosek.** Surowy HTML zawiera tylko 10 z 400 rekordów (2,5%) – pozostałe 390 (97,5%) powstają dopiero w przeglądarce, po kolejnych wywołaniach *fetch* wyzwalanych przewijaniem. Na podstronie surowy HTML nie ma żadnego z atrybutów ze specyfikacji, listy szczegółów i „Read more” (0 elementów wobec 12 w DOM) – to 9 z 33 pozyskanych atrybutów (brand, sku, condition, availability, seller, rating, full_text, author, data publikacji z „Read more”). Parser oparty wyłącznie o *requests* zebrałby więc tylko 2,5% rekordów, i to bez tych 9 atrybutów. Wykryte żądania *fetch*: `/api/v1/feed`, `/api/v1/posts/7/additional-info`, `/api/v1/posts/7/read-more`.

# 7. Czas i pamięć: Selenium vs podejście bez przeglądarki; headless vs okno

Ten sam podzbiór: pierwsze 20 rekordów, jedna przeglądarka, opóźnienie 0,5 s. Pamięć = szczyt RSS procesu Pythona wraz z Chrome i chromedriver.

| Wariant | Czas [s] | Szczyt pamięci [MB] | Żądań HTTP | Uwagi |
|---|---|---|---|---|
| Selenium, okno | 110,7 | 739 | 62 | lista + 2 wywołania API + strona na rekord |
| Selenium, headless | 118,5 | 675 | 62 | jak wyżej, bez okna |
| Bez przeglądarki (JSON) | 1,9 | 83 | 1 | jedno żądanie `/api/v1/feed` |

Przebieg pełny (400 rekordów): Selenium headless, 3 przeglądarki – 864 s (14,4 min), szczyt 1 981 MB, 1241 żądań; endpoint JSON – 11,2 s, 106 MB, 5 żądań.

**Wnioski.** (1) Podejście bez przeglądarki jest ok. 58× szybsze i zużywa ok. 9× mniej pamięci na podzbiorze (w całości ok. 77× szybsze), bo omija renderowanie, wykonywanie skryptów, symulację scrolla i trzy żądania na rekord – pobiera ten sam JSON, z którego korzysta strona, w paczkach po 100 rekordów. Daje przy tym identyczny zbiór. (2) Różnica headless vs okno (111 s vs 118 s) mieści się w szumie pomiaru: czas jest zdominowany przez sztuczne, losowe opóźnienie serwera (0–3 s na żądanie), a nie przez renderowanie; headless oszczędził ok. 9% pamięci. Jedno uruchomienie na wariant – to pomiar orientacyjny, nie statystyczny. (3) Selenium jest jednak konieczne tam, gdzie nie ma publicznego endpointu lub dane powstają w JS; tutaj jest wymagane do spełnienia zadania, a JSON stanowi skrót. Serwis nie ogranicza liczby żądań (nigdy nie zwrócił 429), więc obsługa Retry-After jest zaimplementowana i przetestowana tylko jednostkowo/logicznie, nie na żywo.

# 8. Ograniczenia i założenia

• Kursy walut do PLN są stałe i przybliżone (PLN 1; EUR 4,25; USD 3,85; GBP 5,00; CZK 0,17; JPY 0,026) – służą wyłącznie porównaniu, nie są danymi rynkowymi. • Symbol ¥ interpretowany jako JPY. • Daty dwuznaczne: dd/mm, chyba że wychodzi data z przyszłości; daty względne liczone od dnia scrapingu. • „Unknown” i „Not available” traktowane jako brak. • Testy na lokalnej kopii serwisu partnera; przed scrapingiem cudzych serwisów należy przeczytać robots.txt i regulamin.